



A study of concurrent multi-frontal solvers for modern massively parallel architectures

Jan Trynda^a, Maciej Woźniak^{a,*}, Sergio Rojas^b

^a AGH University of Science and Technology Institute of Computer Science, al. A Mickiewicza 30, 30-059 Kraków, Poland

^b Instituto de Matemáticas, Pontificia Universidad Católica de Valparaíso, Valparaíso, Chile

ARTICLE INFO

Keywords:

Trace theory
Direct solver
Concurrent computations
Matrix factorization

ABSTRACT

Leveraging Trace Theory, we investigate the efficient parallelization of direct solvers for large linear equation systems. Our focus lies on a multi-frontal algorithm, and we present a methodology for achieving near-optimal scheduling on modern massively parallel machines. By employing trace theory with Diekert Graphs and Foata Normal Form, we rigorously validate the correctness of our proposed solution. To establish a strong link between the mesh and elimination tree of the multi-frontal solver, we conduct extensive testing on matrices derived from the Finite Element Method (FEM). Furthermore, we assess the performance of computations on both GPU and CPU platforms, employing practical implementation strategies.

1. Introduction

In modern simulations utilizing methods like the Finite Difference Method (FDM), Finite Element Method (FEM), and others, a two-phase approach is commonly followed. First, the matrices representing the discrete system are obtained, and then the resulting linear or nonlinear algebraic equation system is solved. Solving the systems of linear equations can be accomplished through multifrontal direct solvers such as MUMPS [1–4], SuperLU [5,6], or PaStiX [7], which are known as industry standards. Alternatively, iterative solvers like PCG [8], GMRES [9], or the Additive Schwarz method [10] can be employed.

Iterative solvers offer significant advantages, particularly their high scalability on large clusters, enabling the efficient handling of extensive systems of equations. Compared to direct solvers, iterative solvers typically require fewer computational resources regarding time and memory. However, they do have drawbacks. One major issue is the potential for severe convergence problems, especially when the matrix's conditioning is large and/or efficient preconditioners are unavailable. Additionally, in certain scenarios, iterative solvers may perform worse than direct solvers. This is particularly evident when solving multiple matrices that share the same set of rows and columns, as in mesh-based methods when local grid refinements are performed [11–13].

It is important to note that direct solvers play a vital role within multiple iterative solvers and are essential in various applications. Different solvers are required for specific applications such as elasticity [14], electromagnetism [15], and fluid dynamics [16], air pollution propagation [17,18], as well as different numerical methods. Through the convergence problems associated with iterative solvers,

direct solvers remain valuable in many scenarios. Furthermore, iterative solvers may be slower than direct solvers in certain situations, highlighting the need to carefully consider the choice of solver based on the specific characteristics of the problem at hand.

1.1. State-of-the-art algorithm for direct solving sparse linear systems of equations: Multi-frontal solver

The multi-frontal solver is a widely recognized algorithm for directly solving sparse linear systems of equations. Duff and Reid initially proposed it in their sequential version of the algorithm [19,20], which utilizes the nested dissection algorithm [21]. The core idea behind the multi-frontal solver is to recursively solve the problem by traversing the nodes of the elimination tree.

The elimination tree serves as the input for the algorithm, where the frontal matrices are located in the tree's leaves. The solver algorithm follows the elimination tree from the leaves to the root, solving subproblems at each node. This process is repeated until reaching the root of the elimination tree. Once the root problem is solved, a recursive backward substitution is performed, descending from the root to the leaves, to obtain the final solution.

Among the various direct methods available for solving linear equations, such as LU factorization, QR factorization, and singular value decomposition [22], the multi-frontal solver commonly employs LU factorization, also known as Gaussian elimination. LU factorization decomposes the original matrix A into the product of a lower triangular matrix L and an upper triangular matrix U .

* Corresponding author.

E-mail addresses: macwozni@agh.edu.pl (M. Woźniak), sergio.rojas.h@pucv.cl (S. Rojas).

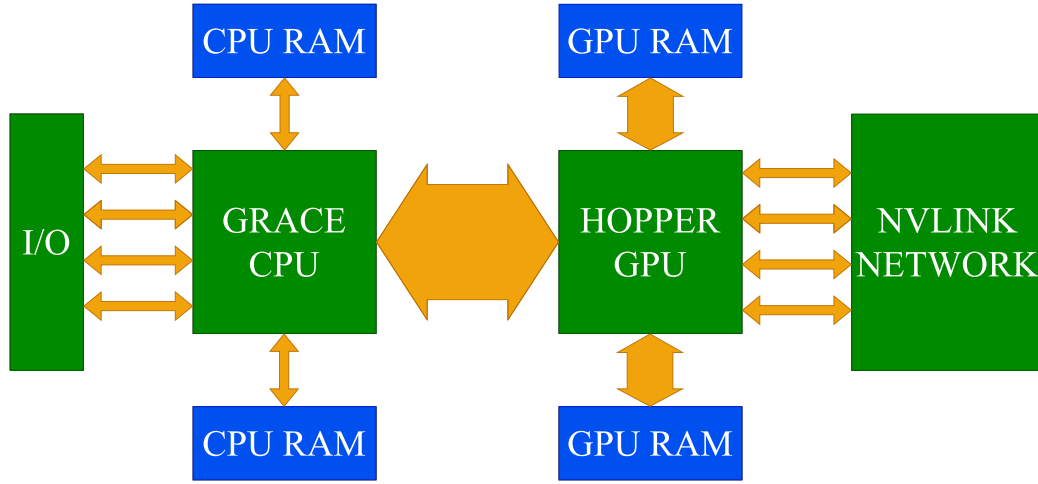


Fig. 1. Simplified of a single Nvidia Grace-Hopper Superchip compute unit. Current state-of-the-art for future supercomputers and data centers.

The fundamental principle of the LU factorization algorithm is to transform the original system of equations into an equivalent system where the coefficient matrix is factored into L and U. This factorization allows for efficient forward and backward substitutions to obtain the solution to the system. The multi-frontal solver incorporates LU factorization as a key step in its solution process, making it a powerful and effective algorithm for directly solving sparse linear systems.

1.2. Schedule of research

Modern state-of-the-art supercomputers, exemplified in Fig. 1, are sophisticated multi-level hierarchical hybrid clusters [23–25]. They consist of nodes (servers) interconnected via high-speed networks, such as Infiniband. Each server can house multiple multi-core CPUs with a shared RAM while accommodating massively parallel co-processors like GPUs. These GPUs possess dedicated memory with a hierarchical structure [26], distinct from the CPU’s memory.

This article focuses on multi-frontal parallel direct solver algorithms designed specifically for modern hybrid memory clusters that feature multiple GPUs per node. Our primary emphasis is on the LU factorization method. While there are alternative algorithms available, such as cuSOLVER or rocSOLVER [27–29], it is worth noting that they lack a formal proof of optimality and do not support cluster configurations.

The trace-theory-based analysis of concurrency presented in this article is not limited to the factorization algorithm alone but can be extended to other contemporary direct factorization methods applicable to general sparse matrix structures. In the interest of clarity and focusing on the trace theory methodology, we have chosen to demonstrate its application in the context of classical FEM, associating the elimination tree with the computational mesh. This simplification allows for a more focused presentation while still showcasing the potential of trace theory in a specific domain. Our approach divides the solver algorithm into multiple indivisible tasks, considering fundamental mathematical operations like summation, multiplication, and more. To establish the execution order of these tasks, we conducted a thorough analysis of their dependencies using trace theory. By introducing dependency relations, we identified classes of tasks that can be executed concurrently. These task classes conform to the Foata Normal Form (FNF), which we achieved through trace theory.

To validate our methodology, we extensively tested the implementation of the solver on an NVIDIA CUDA architecture. This rigorous testing on CUDA ensures the reliability and effectiveness of our approach, demonstrating its suitability for practical applications.

While several articles have explored similar topics, it is essential to note the distinctions. Some of these works have applied Trace Theory to

multi-frontal solver algorithms [12,30,31]. However, they treat operations within a single front as atomic without further decomposing into basic operations such as multiplication or summation. Additionally, these articles often introduce algorithms closely tied to specific graph grammars and FEM computations.

Other works focus on specific types of parallelism, such as shared memory or distributed memory approaches [32–35]. While they contribute valuable insights, they do not address formal concurrency verification specifically for modern massively parallel machines.

It is worth highlighting that, as of now, none of the published works or algorithms formally verify concurrency for modern massively parallel machines. This gap underscores the novelty and significance of our research in providing a formal verification approach that addresses the challenges of concurrency on these advanced computational platforms.

Several other approaches utilized the Directed Acyclic Graph (DAG) technique [36,37]. DAG scheduling results in highly effective real-life applications with relatively fine-grained parallelism. Having praised DAG, it does not provide a strategy nor theoretical limit of parallelization on an “infinite parallel machine”. Trace Theory is, at its core, in many ways similar to DAG. However, the output is different. Besides formal verification of concurrency and scheduling strategy for a GPU cluster, we know concurrency limits on an “ideal, infinite, parallel machine”.

1.3. Structure of the article

The article is organized as follows. Section 2 introduces the background for the further derived methodology. We consider the IGA-FEM matrix obtained from the L^2 -projection problem (this is, the matrix associated with the standard L^2 -inner product) to construct sample sparse matrices. The FEM computational problem simplifies the methodology derivation while providing numerical results. However, we recall our methodology can be applied to any inversion problem involving an invertible symmetric matrix. Section 3 delves into the multi-frontal algorithm and explores how Trace theory is leveraged to develop a concurrent algorithm variant. Section 4 presents the results of several numerical experiments conducted to evaluate the algorithm’s performance. A detailed analysis of the findings is also provided. Lastly, Section 5 offers a comprehensive summary of the paper, highlighting the key contributions and potential future directions for research.

2. Background

2.1. Schur complement

The Schur complement method is a remarkable and widely employed technique for solving linear systems. It provides an efficient

approach to handling such systems by leveraging the inherent properties and structure of the matrices involved. By decomposing the original matrix into submatrices and isolating the Schur complement, which is itself a well-defined matrix, we can simplify the problem and optimize the computational process.

Consider a linear system represented as $Bu = b$, where B is a matrix that can be decomposed into submatrices. Specifically, decomposed into the form:

$$B = \begin{bmatrix} C & D \\ E & -F \end{bmatrix}, \quad (1)$$

where C and F are square matrices of dimensions q and r , respectively. If C is invertible, then the Schur complement method consists of performing a partial LU factorization on the submatrix C , resulting in the following factorization:

$$B = \begin{bmatrix} I & 0 \\ EC^{-1} & I \end{bmatrix} \begin{bmatrix} C & 0 \\ 0 & -(F + EC^{-1}D) \end{bmatrix} \begin{bmatrix} I & C^{-1}D \\ 0 & I \end{bmatrix}. \quad (2)$$

The term $-(F + EC^{-1}D)$ is known as the Schur complement.

In practical applications, matrix C often represents a well-conditioned and computationally efficient system component, while F may pose conditioning or computational cost challenges. By applying the Schur complement method, we can solve the linear system efficiently by first solving for the matrix C and then utilizing this solution to eliminate the dependence on C in the Schur complement matrix. The Schur complement plays a vital role in resolving partial differential equations (PDEs) through residual minimization on dual discrete norms, which is a remarkable example of its application. In a seminal work by authors in [38], it was proven that this type of discretization leads to the inversion of a matrix with a specific structure, as given in Eq. (1), where C is a symmetric and positive definite matrix, $E = D^T$, and $F = 0$. Notably, the Schur complement, in this case, is also a symmetric and positive definite matrix. This key observation has paved the way for the development of various methods, including the DPG method [39–42] (with a more recent overview available in [43]), the AS-FEM [44–47] (along with an isogeometric version discussed in [48]), the iGRM [49–51], among others. These methods have leveraged the properties of the Schur complement to design efficient and robust techniques for solving PDEs, opening up new avenues in computational modeling and simulation.

2.2. Multi-frontal algorithm

The frontal solver algorithm, initially introduced by M. Irons in [52], revolutionized the field of linear systems of equations. This algorithm, rooted in Gaussian elimination, was originally implemented in Fortran. Over time, it gained significant recognition and became a widely adopted standard for industrial and research applications.

I.S. Duff and J.K. Reid pioneered one notable advancement to the frontal solver algorithm [53,54]. Their work introduced the concept of the multi-frontal solver, which proved to be a pivotal modification. The multi-frontal solver was initially devised for indefinite symmetric matrices and later extended to unsymmetric matrices. This modification enabled the simultaneous elimination of multiple fronts, giving rise to the name “multi-frontal”. The multi-frontal solver’s ability to handle complex matrix structures and exploit parallelism has contributed to its widespread adoption and applicability in various domains. Its impact on solving linear systems of equations has been substantial, making it a cornerstone in the field of numerical methods.

The multi-frontal method is a powerful approach for solving systems of linear equations, where $Ax = b$, with A being a sparse matrix and b a vector. This method organizes the matrix factorization operations by dividing the matrix into smaller dense submatrices and performing partial factorization.

The dependencies between these partial factorizations, known as the Schur complement, can be represented as a tree structure (Fig. 2).

To compute a node in the tree, all its child nodes must be computed first, ensuring proper dependencies are maintained. This simultaneous computation of nodes based on their dependencies has been extensively discussed in [53,55,56].

Fig. 2 presents an assembly tree with eight nodes (fronts). Notably, computations in fronts $A1$, $A2$, $A3$, and $A4$ can be performed simultaneously. Furthermore, front $B1$ only requires the completion of fronts $A1$ and $A2$, while front $B2$ requires fronts $A3$ and $A4$. This dependency structure is commonly referred to as the elimination tree.

As the partial elimination progresses, adjacent fronts are merged into larger ones. This allows the processing of incompletely assembled parts of a front at higher levels of the tree. The process is repeated at each level of the tree until reaching the root, where the largest, dense, fully assembled matrix is solved.

The algorithm for the multi-frontal method is presented in Table 1, summarizing the steps involved in the factorization process.

Overall, the multi-frontal method’s effectiveness lies in exploiting parallelism and efficiently handling sparse matrices through a hierarchical factorization approach.

2.3. Elimination tree

In our case, applying the multi-frontal solver has been specifically tailored to the Finite Element Method (FEM), and the Isogeometric Finite Element Method (IGA-FEM). Extensive research, such as the work by Paszyńska et al. [11], has been dedicated to connecting the elimination tree with the computational domain in FEM. To showcase the simplicity of our approach, we generate fronts as elemental matrices.

In the context of our methodology, the assembly of the elimination tree follows a bottom-up approach. The algorithm can be divided into four distinct steps.

1. Generate elemental matrices (fronts) by partitioning the FEM mesh into distinct elements.
2. Assemble the left-hand-side sparse matrix by combining the elemental matrices based on mesh connectivity.
3. Sort the fronts in a specific order that reflects their origin within the mesh.
4. Pair the fronts together to form a binary tree structure, ensuring that the remaining front remains unchanged in the case of an odd number of fronts.
5. Repeat the previous step with the newly created or merged fronts until only the root problem remains.

The assembled elimination tree forms a balanced binary structure, with the potential exception of the farthest right element on each level (excluding the root). Fig. 2 visually represents the elimination tree, where each square represents a front.

The assembly process involves merging smaller fronts from lower levels to form larger fronts. However, there can be instances where this rule does not apply. For instance, a dashed line in the far right section of the tree highlights such an exception. The farthest right element at each level may be formed from two or just one front from a lower level. A simple identity relation exists between the two fronts in the latter case.

By isolating and examining the two-to-one relations, we can focus on the core process of merging smaller fronts to construct the elimination tree.

The composition of the newly assembled front can be described as follows:

- One part that is shared between both of the original fronts. Our attention will be directed toward the yet-to-be-assembled portion of this front during this step of the elimination tree.
- Two parts that are exclusive to only one of the original fronts.
- Two parts that are derived from any previous front, consisting solely of zeros.

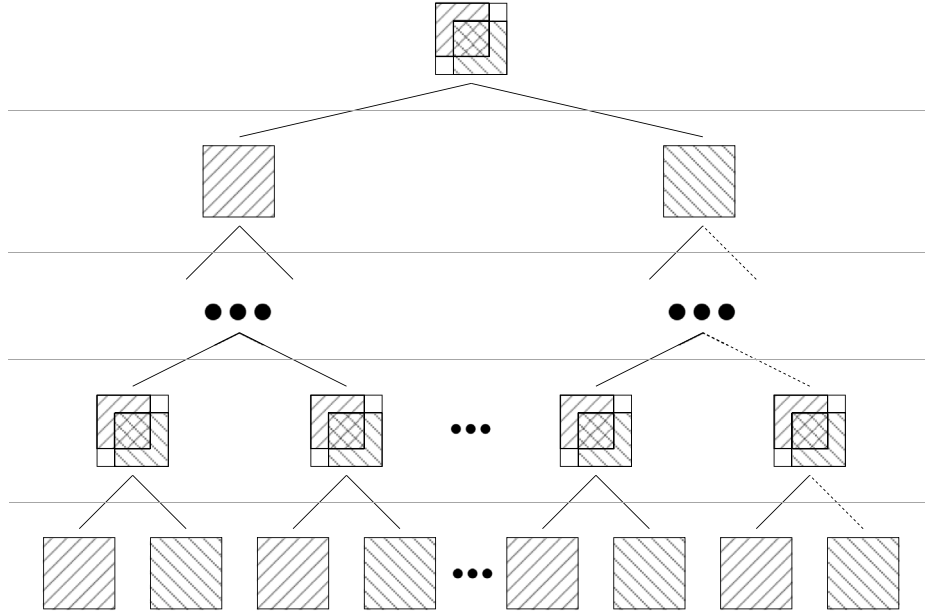


Fig. 2. Visualization of elimination tree.

3. Concurrency model

Formally verifying concurrent computations involves creating a concurrency model using various methods. It requires sophisticated concurrency analysis and meticulous construction of parallel algorithms. To construct the Diekert–Mazurkiewicz trace model, we define an alphabet of tasks and establish a symmetric dependency relation between them. In our model, each task from the alphabet occurs precisely once, ensuring a clear representation of dependencies. To simplify the dependency relation and Diekert Graph, we introduce an additional type of task acting as a barrier. While not directly originating from the simplest algorithm, this task is crucial in streamlining the concurrency analysis. By employing this methodology, we can comprehensively discuss the full concurrency of the multi-frontal solver algorithm. By carefully defining task alphabets, capturing dependencies, and incorporating barrier tasks, we enable a thorough exploration of parallel execution.

The alphabet of tasks for the multi-frontal factorization algorithm consists of the following four tasks. Inside the definition of the tasks, we use z as the global equivalent of index p , y as the global equivalent of index j , and x as the global equivalent of row index i .

1. $T_{\delta;\gamma^1}^{1:f}$ - Division task: Divides a value b_{δ}^f , where $\delta = (p, j)$, and j is the element column index, on the selected pivot row p by the current pivot $b_{\delta^1}^f$, where $\delta^1 = (p, p)$ itself for the current front f . It computes the result of $t_{\delta;\gamma^1}^{1:f} = b_{\delta}^f / b_{\delta^1}^f$. This task is performed once for each element to the right ($j > p$) of the pivot in the same row and the results are used for subsequent multiplication tasks. The γ^1 index points out the location of the elements in the global matrix
2. $T_{\beta;\alpha}^{2:f}$ - Multiplication task: Multiplies the result $t_{\delta;\gamma^1}^{1:f}$ of the corresponding division task $T_{\delta;\gamma^1}^{1:f}$, where $\delta = (p, j)$ by an element $b_{\delta^2}^f$, where $\delta^2 = (i, p)$ and $\beta = (p, i, j)$, where i and j are the elements row indexes of the selected pivot column p for the current pivot $b_{\delta^1}^f$, with $\delta^1 = (p, p)$ and front f . It computes the result of $t_{\beta;\alpha}^{2:f} = t_{\delta;\gamma^1}^{1:f} * b_{\delta^2}^f$. This task is performed once for each element below ($i > p$) the pivot element in the same column. Each result will be used exactly once for subtraction, covered in the next task

type. The index $\alpha = (z, x, y)$ points out the element's location in the global matrix

3. $T_{\beta;\alpha}^{3:f}$ - Subtraction task: Subtracts the results $t_{\beta;\alpha}^{2:f}$ of the corresponding multiplication task $T_{\beta;\alpha}^{2:f}$, where $\alpha = (z, x, y)$, from an element a_{γ} , where $\gamma = (x, y)$ of the global matrix and assigns the newly calculated value to this element. It computes the result of $a_{\gamma} = a_{\gamma} - t_{\beta;\alpha}^{2:f}$. This task is performed once for each multiplication task result
4. T_{γ}^4 - Assertion task: Checks the condition that the element a_{γ} , where $\gamma = (x, y)$ is ready to be used in calculations by verifying if it has reached its final value. This task performs an assertion operation on a_{γ} . It serves to ensure that all assignments to the elements have been completed.

To summarize, each task in the multi-frontal factorization algorithm is represented by one or two upper subscripts, divided into two groups by a semicolon. The first group indicates the task type, while the second group (optional) represents the current front index, denoted as f . Additionally, tasks may have one or two subscripts at the bottom. It is crucial to note that the execution of a specific task depends on completing tasks that provide the required input. Thus, the algorithm ensures that the necessary dependencies are satisfied before executing a particular task.

3.1. Set of dependencies

In this section, we define the alphabet of tasks Σ and the dependencies between them denoted by D . The defined alphabet will consist of the following:

- division tasks for each nonzero element in the pivot row (where $j > p$), for each pivot, for each front,
- multiplication tasks for each nonzero element in the pivot column (where $i > p$), for each division task result,
- subtraction tasks for each multiplication task result,
- assertion tasks for each global matrix element (not only those initially nonzero),

Therefore, a final form of the alphabet will be defined as follows:

$$\Sigma = \{T_{\delta;\gamma^1}^{1:f}, T_{\beta;\alpha}^{2:f}, T_{\beta;\alpha}^{3:f}, T_{\gamma}^4\} \quad (3)$$

Here index f belongs to the set of all fronts in elimination tree. Indexes z , y , and x belong to the sub-matrix of the front f . Now we can describe subsets of dependencies to simplify the construction.

The division depends on two assertions concerning the elements it uses for calculations. Therefore, we can represent those dependencies as follows:

$$J_1 = \{(T_{\gamma^1}^4, T_{\delta:\gamma^1}^{1:f}), (T_{\gamma^2}^4, T_{\delta:\gamma^2}^{1:f})\} \quad (4)$$

Multiplication depends on two tasks. The first task is the corresponding division task that produces the value used during the current calculations. The second task is the assertion that concerns the second element used for the calculations. Therefore, we can represent those dependencies as follows:

$$J_2 = \{(T_{\delta:\gamma^1}^{1:f}, T_{\beta:\alpha}^{2:f}), (T_{\gamma^3}^4, T_{\beta:\alpha}^{2:f})\} \quad (5)$$

Subtraction depends on the corresponding multiplication and multiple subtractions that assign values to the same element a_γ in the global matrix. Therefore, we can represent those dependencies as follows:

$$J_3 = \{(T_{\beta:\alpha}^{2:f}, T_{\beta':\alpha'}^{3:f}), (T_{\beta':\alpha'}^{3:f}, T_{\beta:\alpha}^{3:f})\} \quad (6)$$

where $\alpha' = (z', x, y)$, and $(z' < z)$ or $(z' = z \text{ and } f' < f)$

The assertion depends on all the subtractions that assign values to the same element a_γ in the global matrix. Therefore, we can represent those dependencies as follows:

$$J_4 = \{(T_{\beta':\alpha'}^{3:f}, T_{\gamma}^4)\} \quad (7)$$

where $\alpha' = (z', x, y)$

Finally, we define dependencies between tasks from the alphabet Σ .

$$D = J^+ \cup (J^+)^{-1} \cup I_\Sigma \quad (8)$$

where

$$J = J_1 \cup J_2 \cup J_3 \cup J_4 \quad (9)$$

The primitives described above define the monoid of the traces for the problems under consideration. We denote J as the edges in the Diekert dependency graph [57], which will be illustrated later in Figs. 3, 4, and 5.

After building the primitives of the trace monoid, i.e., the alphabet of tasks (3) and the dependency relation (8), we define the pseudocode allowing us to compute matrix factorization, presented in Table 1. The dependencies recorded in this algorithm determine only the sequence of operations in one string representing the desired trace. By utilizing the task alphabet Σ (3), the dependency relation (8), and executing the pseudocode outlined in Tables 1 and 2, we can calculate the Diekert dependency graph. This graph is helpful for the appropriate and efficient scheduling of tasks in a heterogeneous computing environment.

3.2. Application of trace theory to multi-frontal solver

This section describes the methodology employed to generate the Diekert Dependency Graph (DDG) and the Foata Normal Form (FNF) in the context of the concurrent factorization of sparse matrices using the multi-frontal solver. The DDG comprehensively represents all computational tasks involved in the factorization process, capturing their dependencies. By utilizing the DDG and FNF, we can identify distinct groups of tasks known as Foata classes, which facilitate the practical implementation of concurrent computations.

To simplify the trace and dependency relation construction, we focus on a case where the sparse matrix is directly associated with the FEM or IGA-FEM mesh. The multi-frontal solver algorithm, presented in pseudocode in Table 1, assumes input divided into fronts based on the local contributions of each mesh element to the sparse matrix.

At each level of the elimination tree, the algorithm iterates over each front, merging it towards the tree's root while computing the

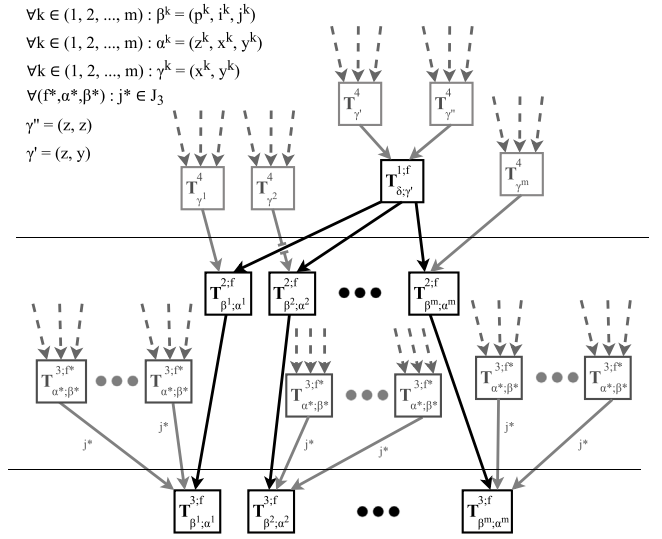


Fig. 3. Main part of the Diekert graph.

Schur complement. It is important to note that certain parts of a front may belong to multiple other fronts simultaneously. In such cases, we select pivots that belong to only one front at a given time. We reorder the front accordingly, placing the pivots to be computed at the current level first, followed by pivots shared by multiple fronts, and finally, pivots computed in previous levels of the elimination tree. This reorganization allows for efficient computations.

The algorithm performs a division task within each pivot's row element. Additionally, a multiplication task and subsequent subtraction task are performed for each element in the pivot column, with the resulting values assigned to the global matrix. Finally, the algorithm computes the remaining front at the root level.

By following this methodology, we can effectively manage the concurrency of tasks in the multi-frontal solver, achieving efficient and accurate factorization of sparse matrices in a concurrent computing environment.

We divided the Diekert graph into multiple recursive sub-graphs. The first sub-graph focuses on operations within a single front, including division, multiplication, and subtraction tasks. Fig. 3 illustrates this sub-graph, with each task represented by a square with a signature. Arrows depict the happen-before relation, denoted as J .

Within this sub-graph, we can identify three types:

1. Sub-graph consisting of a single division task.
2. Sub-graph consisting of multiple multiplication tasks.
3. Sub-graph consisting of multiple subtraction tasks.

The part of the graph involving assertion tasks requires two separate sub-graphs for different cases. Fig. 4 represents the case with an off-diagonal element, while Fig. 5 represents the case with an element on the diagonal. The tasks and relations in these sub-graphs are presented in the same manner as in the previous part of the graph.

Introducing these sub-graphs simplifies the Diekert graph by replacing many-to-many relations with many-to-one relations. Consequently, we have a single relation in the first part of the Diekert graph.

3.3. Scheduling algorithm

To achieve comparable scheduling performance to the classical algorithm on massively parallel shared-memory machines, we adopt the Foata-Normal-Form (FNF) approach [58]. The Diekert dependency graphs (discussed in Section 3.2) illustrate the consecutive Foata classes for each scenario of the multi-frontal solver. Within a particular Foata

Table 1

Multi-frontal solver pseudocode.

```

1  function multi_frontal_solver(fem_input, b_vector):
2      fronts = get_fronts(fem_input)
3
4      # For each elimination tree level
5      while(len(fronts) > 1):
6          # For each front
7          for the front in fronts:
8              # Compute what can be computed on this front
9              compute_front(front)
10         end for
11         # Proceed to higher elimination tree level
12         fronts = join_adjacent_fronts(fronts)
13     end while
14     compute_front(fronts[0])
15     # Final factorized matrix
16     final_matrix = fronts[0]
17
18     # Apply the B vector to return the result
19     return get_solution(final_matrix, b_vector)
20
21
22 function compute_front(front):
23     front = reorder(front)
24     p = 0
25     # As long as the selected pivot is in this front only and has not been computed yet
26     while front[p, p].front_membership_count == 1 and !front[p, p].computed:
27         for j in range(j, len(front)):
28             # Division task
29             t_1 = front[p, j] / front[p, p]
30             for i in range(i, len(front)):
31                 # Multiplication task
32                 t_2 = t_1 * front[i, p]
33                 # Subtraction task
34                 front[i, j].assign(front[i, j] - t_2)
35             end for
36         end for
37         p += 1
38     end while
39     return front

```

Table 2

Elimination tree generation pseudocode.

```

1  function generate_elimination_tree(global_matrix):
2      fronts = divide_into_FEM_based_fronts(global_matrix)
3      fronts = sort_fronts_based_on_lowest_index(fronts)
4      while(len(fronts) > 1):
5          fronts = join_adjacent_fronts(fronts)
6      end while
7      return fronts[0]
8  end function
9
10 function join_adjacent_fronts(fronts):
11     new_fronts = []
12     for i in range(0, len(fronts) - 1, step=2):
13         new_fronts.append(join_fronts(fronts[i], fronts[i+1]))
14     end for
15     if len(fronts)%2 == 1:
16         new_fronts.append(fronts[len(fronts) - 1])
17     end if
18     return new_fronts
19 end function

```

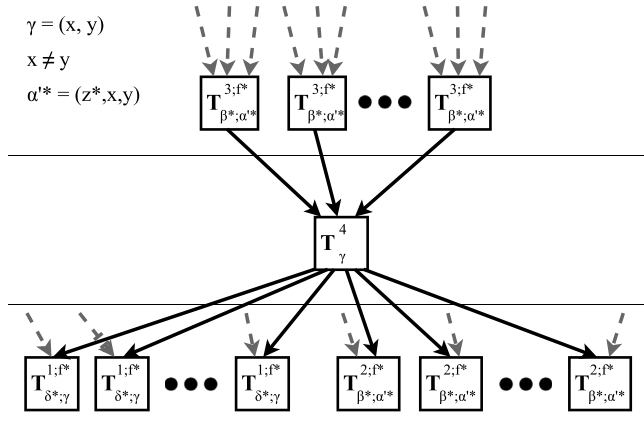


Fig. 4. Diekert graph part focused on a type 4 task regarding element off the diagonal.

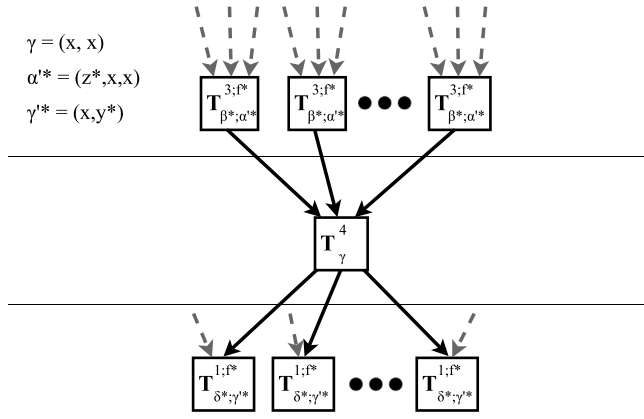


Fig. 5. Diekert graph part focused on a type 4 task regarding element on the diagonal.

class, the tasks can be executed in any order, and the completion of the entire preceding Foata class is sufficient to initiate the computation of the next class. This strategy ensures the absence of deadlocks, facilitates high-quality scheduling, and eliminates the need for intra-class synchronization.

To simplify the construction of the FNF, we first define groups of tasks that always belong to the same FNF class. These task groups serve as a means to streamline the definition of FNF classes and enhance the overall organization of the algorithm.

For each front (f), each pivot, and each element in the pivot row (δ, γ^1 - global index), we can define three groups of tasks:

$$p_{\delta; \gamma^1}^{1; f} = \{T_{\delta; \gamma^1}^{1; f}\} \quad (10)$$

$$p_{\delta; \gamma^1}^{2; f} = \{T_{\beta^1; \alpha^1}^{2; f}, T_{\beta^2; \alpha^2}^{2; f}, \dots, T_{\beta^m; \alpha^m}^{2; f}\}, \forall l \in (1, 2, \dots, m) : \beta^l = p, i^l, j \quad (11)$$

$$p_{\delta; \gamma^1}^{3; f} = \{T_{\beta^1; \alpha^1}^{3; f}, T_{\beta^2; \alpha^2}^{3; f}, \dots, T_{\beta^m; \alpha^m}^{3; f}\} \quad (12)$$

We should also define an additional group of tasks for each global matrix element. This is a synchronization/barrier group.

$$p_{\gamma}^4 = \{T_{\gamma}^4\} \quad (13)$$

When defining the task groups, we can assume that each task belongs to exactly one group, corresponding to a single Foata class. The Foata classes will be constructed by merging multiple task groups together. For example, the tasks $p_{\delta; \gamma^1}^{1; f}$, $p_{\delta; \gamma^1}^{2; f}$, and $p_{\delta; \gamma^1}^{3; f}$ will be part of consecutive Foata classes in that specific order.

Based on Figs. 3 through 5, we can outline a general procedure for creating subsequent Foata classes, which involves the following tasks.

- Class q_1 , where $q \in \{1, 5, 9, \dots, 4w + 1\}$

$$\{p_{\gamma^*}^4, p_{\gamma^*}^4, \dots, p_{\gamma^*}^4\} \quad (14)$$

- Class q_2 , where $q \in \{2, 6, 10, \dots, 4w + 2\}$

$$\{p_{\delta^*; \gamma^*}^{1; f^*}, p_{\delta^*; \gamma^*}^{1; f^*}, \dots, p_{\delta^*; \gamma^*}^{1; f^*}\} \quad (15)$$

- Class q_3 , where $q \in \{3, 7, 11, \dots, 4w + 3\}$

$$\{p_{\delta^*; \gamma^*}^{2; f^*}, p_{\delta^*; \gamma^*}^{2; f^*}, \dots, p_{\delta^*; \gamma^*}^{2; f^*}\} \quad (16)$$

- Class q_4 , where $q \in \{4, 8, 12, \dots, 4w + 4\}$

$$\{p_{\delta^*; \gamma^*}^{3; f^*}, p_{\delta^*; \gamma^*}^{3; f^*}, \dots, p_{\delta^*; \gamma^*}^{3; f^*}\} \quad (17)$$

Here, $*$ represents a placeholder for any (multi-)index value. The notation used does not consistently refer to the same value, resulting in potential discrepancies. For instance, when considering an (multi-)index w , the value represented by w^* in one equation may not be

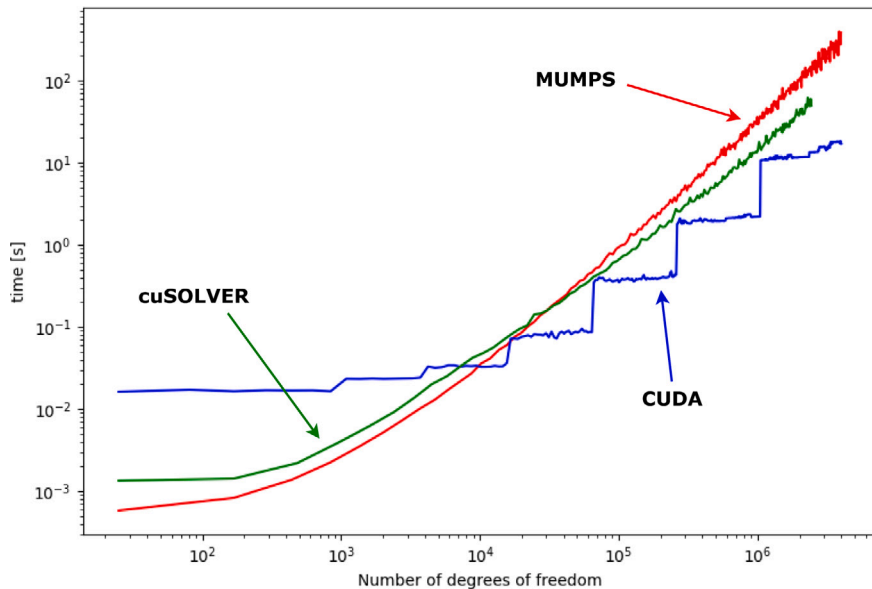


Fig. 6. Comparison of CPU and GPU factorization time of sparse matrices for linear basis functions. MUMPS compiled with gcc.

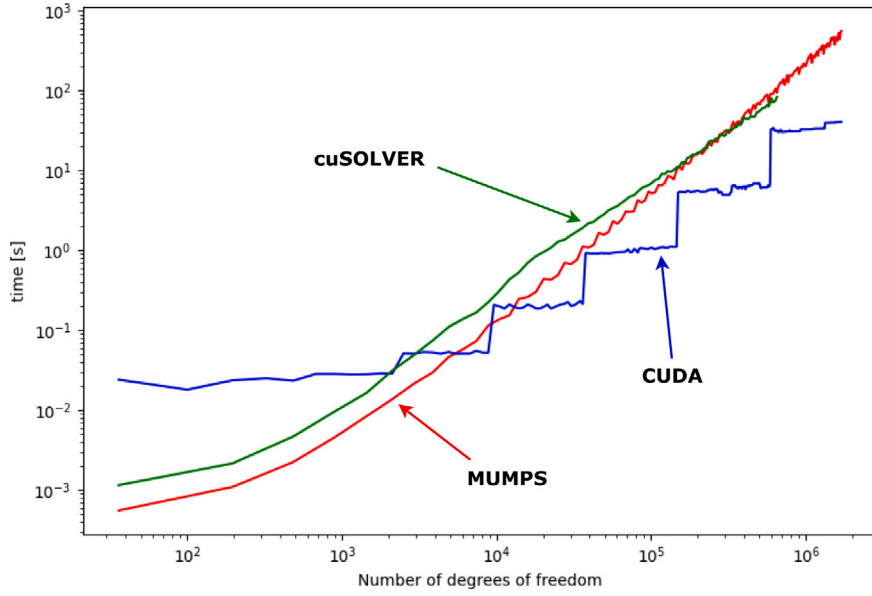


Fig. 7. Comparison of CPU and GPU factorization time of sparse matrices for quadratic basis functions. MUMPS compiled with gcc.

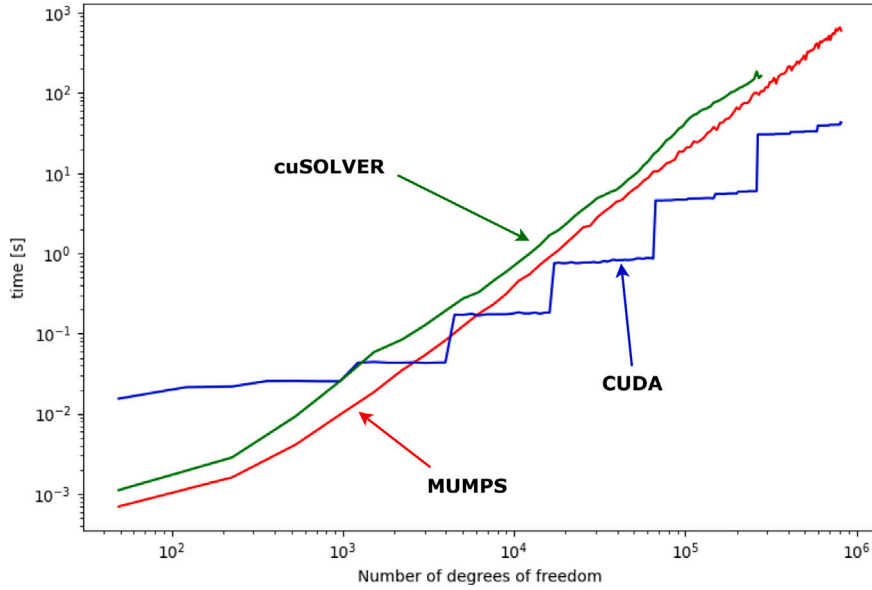


Fig. 8. Comparison of CPU and GPU factorization time of sparse matrices for cubic basis functions. MUMPS compiled with gcc.

equivalent to the value represented by w^* in other sections of the same or different equation.

We know that $\forall k$ if class k takes the form

$$\{p_{\delta^*, \gamma^*}^{1:f^*}, p_{\delta^*, \gamma^*}^{1:f^*}, \dots, p_{\delta^*, \gamma^*}^{1:f^*}\} \quad (18)$$

then, the two following classes will have the form of

- Class $k + 1$

$$\{p_{\delta^*, \gamma^*}^{2:f^*}, p_{\delta^*, \gamma^*}^{2:f^*}, \dots, p_{\delta^*, \gamma^*}^{2:f^*}\} \quad (19)$$

- Class $k + 2$

$$\{p_{\delta^*, \gamma^*}^{3:f^*}, p_{\delta^*, \gamma^*}^{3:f^*}, \dots, p_{\delta^*, \gamma^*}^{3:f^*}\} \quad (20)$$

where, if and only if, class k contains element $p_{\delta^*, \gamma^*}^{1:f^*}$, then class $k + 1$ contains element $p_{\delta^*, \gamma^*}^{2:f^*}$ and class $k + 2$ contains element $p_{\delta^*, \gamma^*}^{3:f^*}$.

The first type of Foata classes (14) is responsible for assertions/synchronization. Subsequent Foata classes follow this:

- The second type of Foata classes (15) is responsible for divisions tasks.
- The third type of Foata classes (16) is responsible for multiplication based on results from the preceding second type Foata class (15).
- The fourth type of Foata classes (17) is responsible for subtraction based on results from the preceding third type Foata class (16).

All the mentioned tasks are executed on a uniform architecture, which allows us to anticipate similar execution times for each task within a specific Foata class. This uniformity enables the efficient scheduling of tasks within a Foata class using a simple bag-of-tasks scheduling method.

We start the computation with Foata class 1 and ensure that all tasks within the class are completed before moving on to the next Foata class. This sequential execution of Foata classes guarantees the correct order of computations and dependencies.

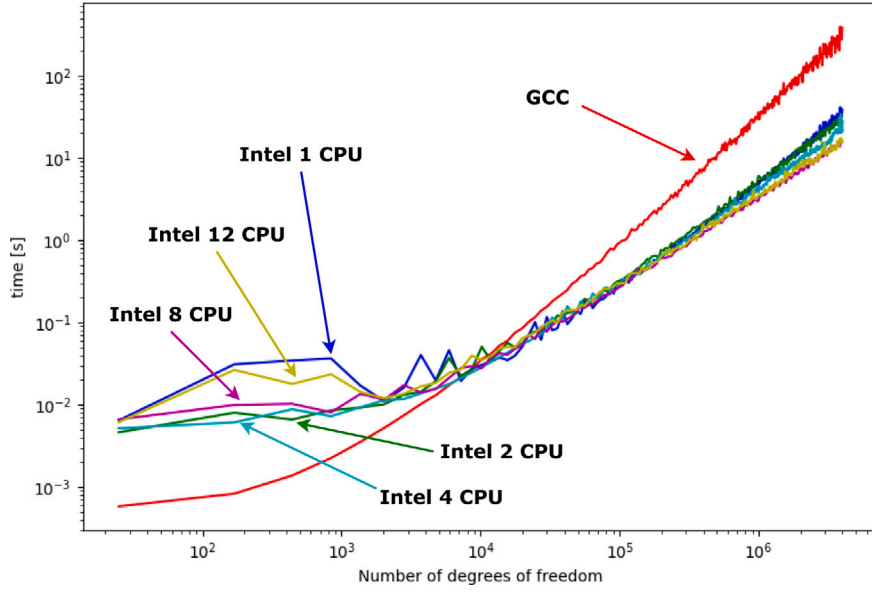


Fig. 9. Comparison of CPU factorization time of sparse matrices for linear basis functions. MUMPS compiled with gcc, and MUMPS compiled with oneAPI.

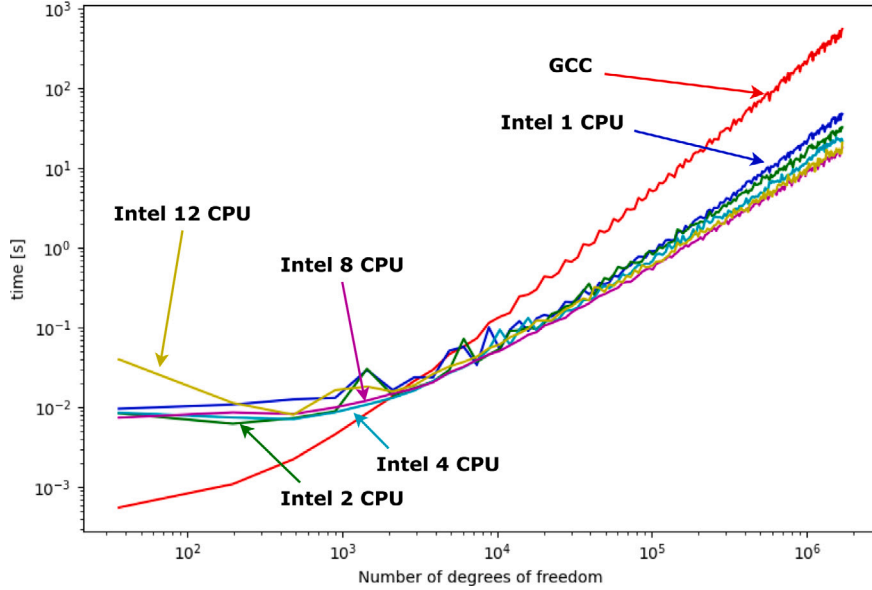


Fig. 10. Comparison of CPU factorization time of sparse matrices for quadratic basis functions. MUMPS compiled with gcc, and MUMPS compiled with oneAPI.

By adopting this simplified scheduling approach based on the Foata Normal Form (FNF), we achieve optimal performance on theoretical “infinite” GPU. The performance is near-optimal in practical applications, even though no theoretical proof exists to support it. The larger (more parallel) the target machine, the closer the implementation gets to optimal. This approach balances efficiency and simplicity, allowing for straightforward implementation and effective utilization of computational resources.

4. Numerical results

We conducted computational performance evaluations of the proposed algorithm on both CPU and GPU architectures. Three modes of computation were considered: single-core CPU, multi-core CPU, and GPU. The execution time was measured for the sequential factorization algorithm on the CPU and the concurrent integration algorithm on a shared memory CPU with twelve cores (24 threads). The experiments

were conducted on a Banach Linux workstation with an AMD Ryzen 9 3900X, 64 GB of RAM memory, and Nvidia GTX 2080 Super GPU.

All tests were performed using sparse matrices from a 2D IGA-FEM L^2 -projection problem. The matrices were generated using external software and subsequently tested with two different implementations of sparse matrix solvers. The CPU implementation was MUMPS compiled with OpenMP enabled. Here, we considered two configurations. In the first one, MUMPS is linked with standard BLAS and LAPACK, and everything is compiled with the Gnu Compiler Collection. In the second one, we utilized MKL, which was compiled considering Intel oneAPI compilers. The GPU version was implemented in CUDA. We also included cuSOLVER [27] as a benchmark implementation for GPU.

Performance measurements were obtained for various mesh sizes and polynomial degrees. The reported time results are for $p = 1$ (Fig. 6), $p = 2$ (Fig. 7), and $p = 3$ (Fig. 8). Our experiments were limited to 3×10^6 degrees of freedom, as the memory was a significant limitation. Domain decomposition should allow for much more significant problems.

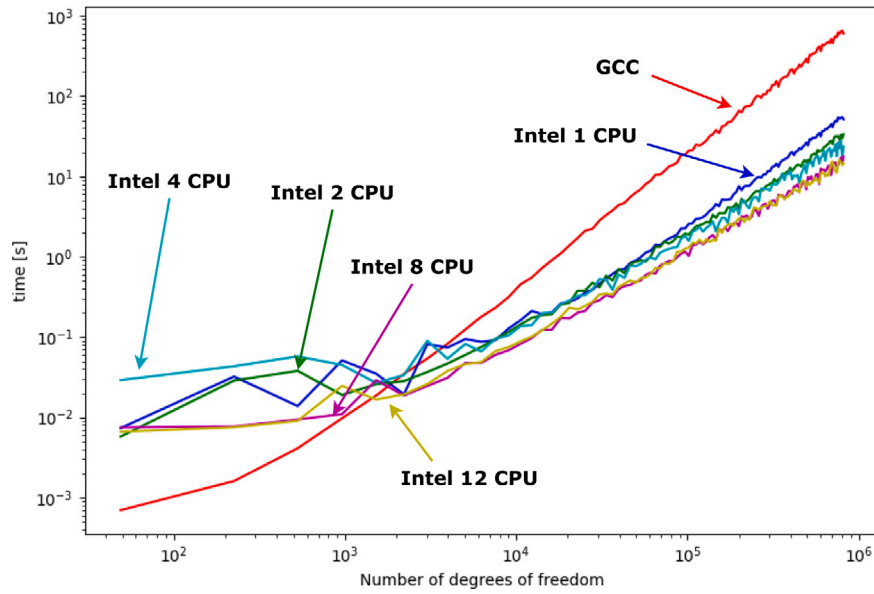


Fig. 11. Comparison of CPU factorization time of sparse matrices for cubic basis functions. MUMPS compiled with gcc, and MUMPS compiled with oneAPI.

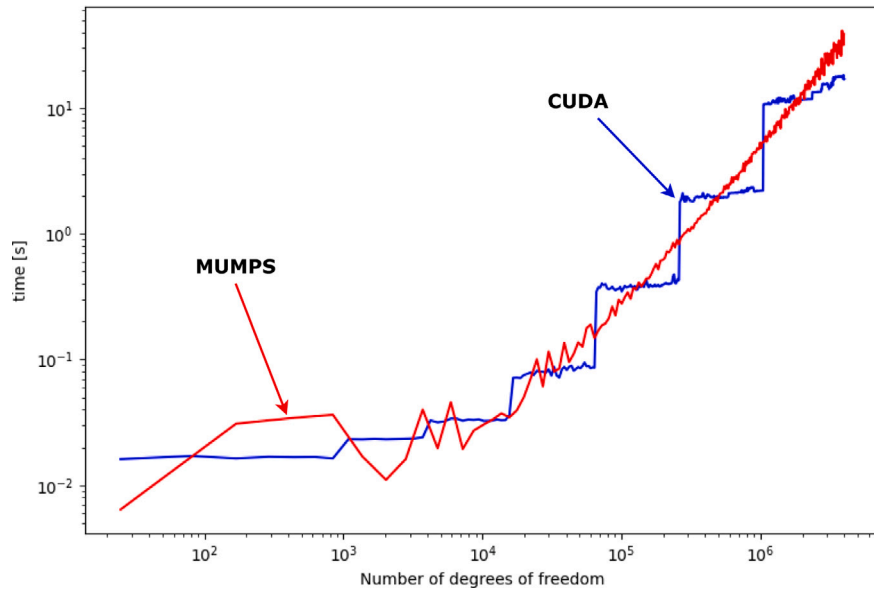


Fig. 12. Comparison of CPU, and GPU factorization time of sparse matrices for linear basis functions. MUMPS compiled with oneAPI.

However, as our research aims at general sparse matrix factorization, we decided not to do that.

The comparison shown in Figs. 6–8 highlights the substantial speed advantage of sparse matrix factorization on a shared memory graphics card, surpassing the integration on CPU by one to two orders of magnitude.

Next, we compared factorization time between different configurations of MUMPS. The one compiled with GCC, with OpenMP enabled and linked with standard (BLAS, LAPACK) libraries, did not speed up on multicore CPU. While the one compiled with Intel oneAPI and linked with MKL showed some scalability on multicore CPU. Even with the same optimization options for both compilers, using oneAPI resulted in around 7.5 times faster execution across the board, also for the integration part of the code. This nearly matches eight times the speedup from using the AVX vector instruction set. The performance measurements were obtained for various mesh sizes and polynomial degrees. The reported time results are for $p = 1$ (Fig. 9), $p = 2$ (Fig. 10), and $p = 3$ (Fig. 11). Our experiments were limited to 3×10^6 degrees of

freedom, as the memory was a significant limitation. In all experiments, we were using shared memory configuration with OpenMP.

Finally, we compared factorization time between MUMPS compiled with Intel oneAPI, linked with MKL, and executed on a single core of CPU with our solution for the GPU. The performance measurements were obtained for various mesh sizes and polynomial degrees. The reported time results are for $p = 1$ (Fig. 12), $p = 2$ (Fig. 13), and $p = 3$ (Fig. 14).

The comparison shown in Figs. 6–8 highlights sections of constant factorization time for our solution. The constant cost sections are strictly related to the amount of CUDA cores available on a GPU. In other words, our method of scheduling remains at constant cost until we exceed the available amount of cores.

When increasing the density of the matrix, memory transfers start to play a bigger role in computations. When considering denser matrices, constant cost sections get shorter. When moving to 3D, they get shorter even faster. To our surprise, when running cuSOLVER (part of

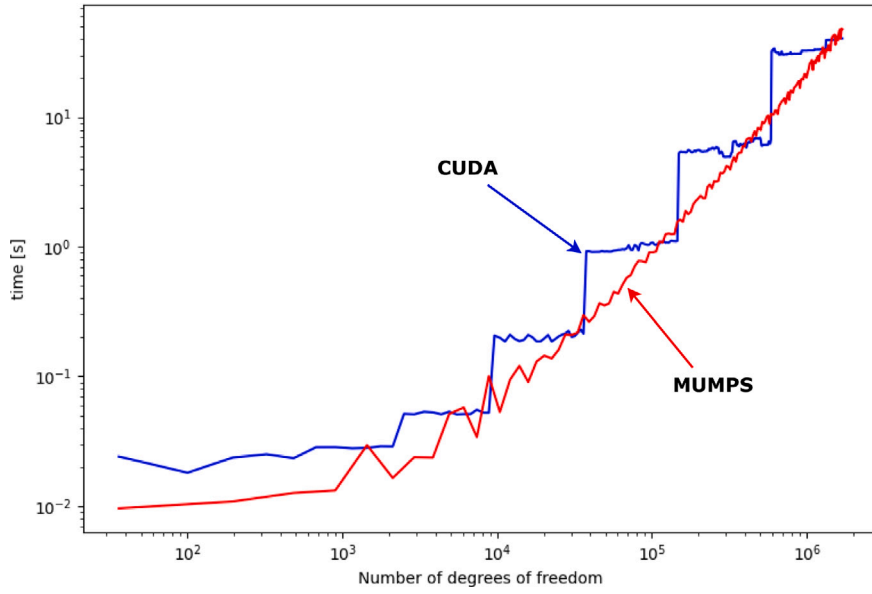


Fig. 13. Comparison of CPU, and GPU factorization time of sparse matrices for quadratic basis functions. MUMPS compiled with oneAPI.

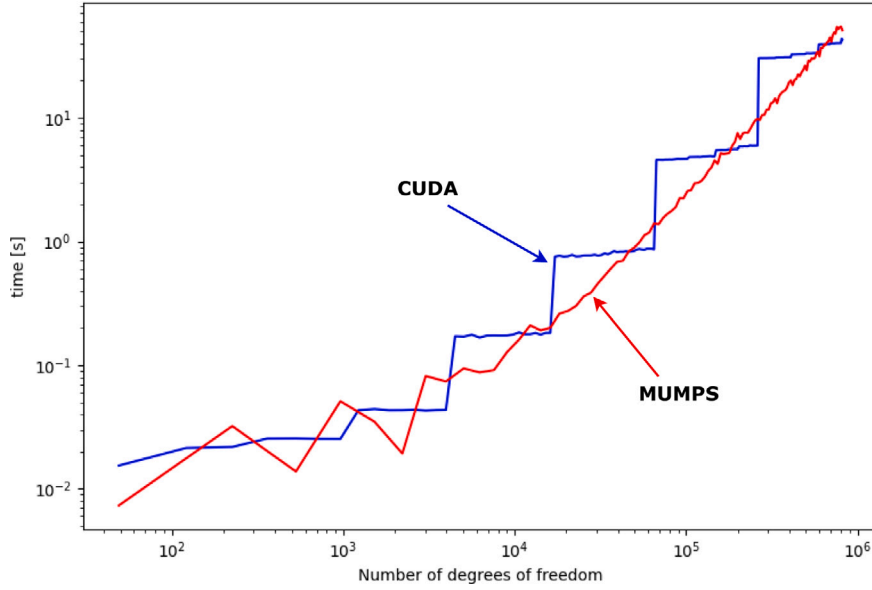


Fig. 14. Comparison of CPU, and GPU factorization time of sparse matrices for cubic basis functions. MUMPS compiled with oneAPI.

cuSPARSE library) implementation, we ended with nearly identical performance as MUMPS, and our algorithm surpassed cuSOLVER between one and two orders of magnitude. Furthermore, our solution used less memory than cuSOLVER, and we were able to solve larger problems.

Our implementation has been optimized for optimal performance, using our knowledge and expertise in optimizing summations and reductions. However, it is worth mentioning that our current implementation does not support multi-GPU or multi-node configurations. Despite this limitation, we can still estimate scalability in such scenarios.

Adding more computational nodes may lead to efficiency drops due to communication costs, particularly in NUMA architectures [59,60]. The extent of these efficiency drops can vary significantly depending on the specific multi-node, multi-GPU architecture employed.

5. Conclusions

We have developed a trace theory-based parallel multi-frontal direct solver, which provides efficient solutions to linear systems of equations.

The solver algorithm is expressed as a sequence of atomic tasks, each with specific dependencies between them. We have identified the task alphabet and dependency relations for the sequential trace applied to the Finite Element Method.

Our methodology enables near-optimal scheduling on modern parallel architectures, leveraging the Foata Normal Form. The solver execution involves multiple Foata classes, where independent tasks are computed concurrently within each class. Synchronization barriers are employed between subsequent classes to ensure correct dependencies. We tested an NVIDIA Tesla GPU and a multicore CPU to evaluate the solver's performance. The methodology was validated using a simple L^2 -projection problem for the Finite Element Method, but its applicability extends to any sparse matrix.

Looking ahead, future work can explore the application of our methodology to create state-of-the-art implementations for cutting-edge machines, such as the recently introduced NVIDIA Grace Hopper architecture [61]. By leveraging the trace theory-based approach, we

can further enhance the solver's efficiency and scalability, enabling it to tackle complex problems on advanced computing platforms.

Our methodology and mathematical model of concurrency deliver theoretical limits of concurrency for sparse matrix solvers. In other words, the algorithm will not surpass specific performance on ideal, "infinite" parallel machines.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgments

The work of MW was supported by program "Excellence initiative – research university" for the AGH University of Science and Technology. The visit of SR in AGH was supported by The European Union's Horizon 2020 Research and Innovation Program of the Marie Skłodowska-Curie grant agreement No. 777778, MATHROCKs. The work of SR was partially supported by the Chilean grant ANID FONDECYT No. 3210009.

References

- [1] P.R. Amestoy, I.S. Duff, J.-Y. L'Excellent, Multifrontal parallel distributed symmetric and unsymmetric solvers, *Comput. Methods Appl. Mech. Engrg.* 184 (2000) 501–520.
- [2] P.R. Amestoy, I.S. Duff, J. Koster, J.-Y. L'Excellent, A fully synchronous multi-frontal solver using distributed dynamic scheduling, *SIAM J. Matrix Anal. Appl.* 23 (2001) 15–41.
- [3] P.R. Amestoy, A. Guermouche, J.-Y. L'Excellent, S. Pralet, Hybrid scheduling for the parallel solution of linear systems, *Parallel Comput.* 32 (2006) 136–156.
- [4] MUMPS Users' guide, http://mumps.enseiht.fr/doc/userguide_5.0.2.pdf.
- [5] X.S. Li, An overview of superlu: Algorithms, implementation, and user interface, *TOMS Trans. Math. Softw.* 31 (2005) 302–325.
- [6] X. Li, J. Demmel, J. Gilbert, i. Grigori, M. Shao, I. Yamazaki, SuperLU Users' Guide, Lawrence Berkeley National Laboratory, 1999, <http://crd.lbl.gov/~xiaoye/SuperLU/>.
- [7] P. Hénon, P. Ramet, J. Roman, Pastix: A high-performance parallel direct solver for sparse symmetric definite systems, *Parallel Comput.* 28 (2002) 301–321.
- [8] C.T. Kelley, *Iterative Methods for Linear and Nonlinear Equations*, Society for Industrial and Applied Mathematics, 1995.
- [9] K. Banaś, M.F. Wheeler, Preconditioning GMRES for discontinuous Galerkin approximations, *Comput. Assist. Mech. Eng. Sci.* 11 (1) (2014) 47–62.
- [10] B. Smith, P. Björstad, W. Gropp, *Parallel Multilevel Methods for Elliptic Partial Differential Equations*, Cambridge University Press, 2004.
- [11] A. Paszyńska, M. Paszyński, K. Jopek, M. Woźniak, D. Goik, P. Gurgul, H. AbouEisha, M. Moshkov, V.M. Calo, A. Lenharth, et al., Quasi-optimal elimination trees for 2D grids with singularities, *Sci. Program.* 2015 (2015) 4.
- [12] D. Goik, K. Jopek, M. Paszyński, A. Lenharth, D. Nguyen, K. Pingali, Graph grammar based multi-thread multi-frontal direct solver with galois scheduler, *Procedia Comput. Sci.* 29 (2014) 960–969.
- [13] H. AbouEisha, M. Moshkov, V. Calo, M. Paszynski, D. Goik, K. Jopek, Dynamic programming algorithm for generation of optimal elimination trees for multi-frontal direct solver over h-refined grids, *Procedia Comput. Sci.* 29 (2014) 947–959.
- [14] A. El Maliki, M. Fortin, N. Tardieu, A. Fortin, Iterative solvers for 3D linear and nonlinear elasticity problems: Displacement and mixed formulations, *Internat. J. Numer. Methods Engrg.* 83 (13) (2010) 1780–1802.
- [15] R. Hiptmair, Multigrid method for Maxwell's equations, *SIAM J. Numer. Anal.* 36 (1) (1998) 204–225.
- [16] D.N. Arnold, R.S. Falk, R. Winther, Multigrid in H (div) and H (curl), *Numer. Math.* 85 (2) (2000) 197–217.
- [17] A. Oliver, G. Montero, R. Montenegro, E. Rodríguez, J. Escobar, A. Pérez-Foguet, Adaptive finite element simulation of stack pollutant emissions over complex terrains, *Energy* 49 (2013) 47–60.
- [18] K. Podsiadło, A.O. Serra, A. Paszyńska, R. Montenegro, I. Henriksen, M. Paszyński, K. Pingali, Parallel graph-grammar-based algorithm for the longest-edge refinement of triangular meshes and the pollution simulations in lesser Poland area, *Eng. Comput.* (2021) 1–24.
- [19] I.S. Duff, J.K. Reid, The multifrontal solution of unsymmetric sets of linear equations, *SIAM J. Sci. Stat. Comput.* 5 (3) (1984) 633–641.
- [20] I.S. Duff, J.K. Reid, The multifrontal solution of indefinite sparse symmetric linear, *ACM Trans. Math. Softw.* 9 (3) (1983) 302–325.
- [21] M.S. Khair, G.L. Miller, T.J. Sheffler, *Nested Dissection: A Survey and Comparison of Various Nested Dissection Algorithms*, Carnegie-Mellon University. Department of Computer Science, 1992.
- [22] C.F. Van Loan, G. Golub, *Matrix computations* (Johns Hopkins studies in mathematical sciences), *Matrix Comput.* 53 (1996).
- [23] Cyfronet KDM, <https://kdm.cyfronet.pl/portal/Main.page>.
- [24] Stampede2 User Guide, <https://portal.tacc.utexas.edu/user-guides/stampede2>.
- [25] Summit, Oak Ridge National Laboratory, <https://www.olcf.ornl.gov/summit/>.
- [26] CUDA Toolkit Documentation, programming guide, <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>.
- [27] cuSOLVER API Reference, <https://docs.nvidia.com/cuda/cusolver>.
- [28] rocSOLVER manual, <https://rocsolver.readthedocs.io/en/rocm-5.5.0/>.
- [29] P. Ghysels, R. Synk, High performance sparse multifrontal solvers on modern GPUs, *Parallel Comput.* 110 (2022) 102897.
- [30] K. Kuźnik, M. Paszyński, V. Calo, Graph grammar-based multi-frontal parallel direct solver for two-dimensional isogeometric analysis, *Procedia Comput. Sci.* 9 (2012) 1454–1463.
- [31] P. Obrok, P. Pierzchała, A. Szymczak, M. Paszyński, Graph grammar-based multi-thread multi-frontal parallel solver with trace theory-based scheduler, *Procedia Comput. Sci.* 1 (1) (2010) 1993–2001.
- [32] M. Woźniak, M. Paszyński, D. Pardo, L. Dalcin, V.M. Calo, Computational cost of isogeometric multi-frontal solvers on parallel distributed memory machines, *Comput. Methods Appl. Mech. Engrg.* 284 (2015) 971–987.
- [33] M. Paszyński, D. Pardo, A. Paszyńska, Parallel multi-frontal solver for p adaptive finite element modeling of multi-physics computational problems, *J. Comput. Sci.* 1 (1) (2010) 48–54.
- [34] M. Woźniak, K. Kuźnik, M. Paszyński, V.M. Calo, D. Pardo, Computational cost estimates for parallel shared memory isogeometric multi-frontal solvers, *Comput. Math. Appl.* 67 (10) (2014) 1864–1883.
- [35] M.M. Łoś, M. Woźniak, M. Paszyński, A. Lenharth, M.A. Hassaan, K. Pingali, IGA-ADS: Isogeometric analysis FEM using ADS solver, *Comput. Phys. Comm.* 217 (2017) 99–116.
- [36] I. Duff, F. Lopez, S. Nakov, Sparse direct solution on parallel computers, in: *Numerical Analysis and Optimization: NAO-IV*, Muscat, Oman, January 2017 NAO-IV, Springer, 2018, pp. 67–98.
- [37] K. Kim, V. Eijkhout, A parallel sparse direct solver via hierarchical DAG scheduling, *ACM Trans. Math. Software* 41 (1) (2014) <http://dx.doi.org/10.1145/2629641>.
- [38] A. Cohen, W. Dahmen, G. Welper, Adaptivity and variational stabilization for convection-diffusion equations, *M2AN* 46 (5) (2012) 1247–1273.
- [39] L. Demkowicz, J. Gopalakrishnan, A class of discontinuous Petrov–Galerkin methods. Part I: the transport equation, *CMAME* 199 (2010) 1558–1572.
- [40] L. Demkowicz, J. Gopalakrishnan, A class of discontinuous Petrov–Galerkin methods. II. Optimal test functions, *NMPDE* 27 (2011) 70–105.
- [41] L. Demkowicz, J. Gopalakrishnan, A.H. Niemi, A class of discontinuous Petrov–Galerkin methods. Part III: Adaptivity, *Appl. Numer. Math.* 62 (4) (2012) 396–427.
- [42] J. Zitelli, I. Muga, L. Demkowicz, J. Gopalakrishnan, D. Pardo, V.M. Calo, A class of discontinuous Petrov–Galerkin methods. Part IV: The optimal test norm and time-harmonic wave propagation in 1D, *J. Comput. Phys.* 230 (7) (2011) 2406–2432.
- [43] L. Demkowicz, J. Gopalakrishnan, An overview of the discontinuous Petrov Galerkin method, in: X. Feng, O. Karakashian, Y. Xing (Eds.), *Recent Developments in Discontinuous Galerkin Finite Element Methods for Partial Differential Equations: 2012 John H Barrett Memorial Lectures*, in: *The IMA Volumes in Mathematics and its Applications*, vol. 157, Springer, Cham, 2014, pp. 149–180.
- [44] V.M. Calo, A. Ern, I. Muga, S. Rojas, An adaptive stabilized conforming finite element method via residual minimization on dual discontinuous Galerkin norms, *Comput. Methods Appl. Mech. Engrg.* 363 (2020) 112891.
- [45] R.J. Cier, S. Rojas, V.M. Calo, Automatically adaptive, stabilized finite element method via residual minimization for heterogeneous, anisotropic advection–diffusion–reaction problems, *Comput. Methods Appl. Mech. Engrg.* 385 (2021) 114027.
- [46] S. Rojas, D. Pardo, P. Behnoudfar, V.M. Calo, Goal-oriented adaptivity for a conforming residual minimization method in a dual discontinuous Galerkin norm, *Comput. Methods Appl. Mech. Engrg.* 377 (2021) 113686.
- [47] F.E. Kyburg, S. Rojas, V.M. Calo, Incompressible flow modeling using an adaptive stabilized finite element method based on residual minimization, *Internat. J. Numer. Methods Engrg.* 123 (8) (2022) 1717–1735.
- [48] M. Łoś, S. Rojas, M. Paszyński, I. Muga, V.M. Calo, DGIRM: Discontinuous Galerkin based isogeometric residual minimization for the Stokes problem, *J. Comput. Sci.* 50 (2021) 101306.
- [49] V. Calo, M. Łoś, Q. Deng, I. Muga, M. Paszyński, Isogeometric residual minimization method (iGRM) with direction splitting preconditioner for stationary advection-dominated diffusion problems, *Comput. Methods Appl. Mech. Engrg.* 373 (2021) 113214.

- [50] M. Łoś, J. Muñoz-Matute, I. Muga, M. Paszyński, Isogeometric residual minimization method (iGRM) with direction splitting for non-stationary advection–diffusion problems, *Comput. Math. Appl.* 79 (2) (2020) 213–229.
- [51] M. Łoś, I. Muga, J. Muñoz-Matute, M. Paszyński, Isogeometric residual minimization (iGRM) for non-stationary Stokes and Navier–Stokes problems, *Comput. Math. Appl.* 95 (2021) 200–214, Recent Advances in Least-Squares and Discontinuous Petrov–Galerkin Finite Element Methods.
- [52] B.M. Irons, A frontal solution program for finite element analysis, *Internat. J. Numer. Methods Engrg.* 2 (1) (1970) 5–32.
- [53] I.S. Duff, J.K. Reid, The multifrontal solution of indefinite sparse symmetric linear, *ACM Trans. Math. Software* 9 (1983) 302–325.
- [54] I.S. Duff, J.K. Reid, The multifrontal solution of unsymmetric sets of linear equations, *SIAM J. Sci. Stat. Comput.* 5 (1984) 633–641.
- [55] M. Paszyński, D. Pardo, C. Torres-Verdín, L. Demkowicz, V. Calo, A parallel direct solver for the self-adaptive hp finite element method, *J. Parallel Distrib. Comput.* 70 (3) (2010) 270–281.
- [56] P. Geng, J.T. Oden, R.A. Van de Geijn, A parallel multifrontal algorithm and its implementation, *Comput. Methods Appl. Mech. Engrg.* 149 (1–4) (1997) 289–301.
- [57] A. Mazurkiewicz, Concurrent program schemes and their interpretations, *DAIMI Rep. Ser.* 78 (1977).
- [58] V. Diekert, Y. Métivier, Partial commutation and traces, *Handb. Formal Lang.* (1997) 457–533.
- [59] K. Zhang, H. Su, Y. Dou, Multilevel parallelism optimization of stencil computations on simdized NUMA architectures, *J. Supercomput.* 77 (2021) 13584–13600.
- [60] H. Dursun, M. Kunaseth, K.-i. Nomura, J. Chame, R.F. Lucas, C. Chen, M. Hall, R.K. Kalia, A. Nakano, P. Vashishta, Hierarchical parallelization and optimization of high-order stencil computations on multicore clusters, *J. Supercomput.* 62 (2012) 946–966.
- [61] NVidia Grace Hopper Superchip, <https://www.nvidia.com/en-us/data-center/grace-hopper-superchip/>.



Jan Trynda graduated in 2022 with Engineering degree at Computer Science at AGH University of Science and Technology, Kraków, Poland. His research interest includes optimization of algorithms for different architectures of parallel machines, isogeometric analysis as well as formal models of concurrency.



Maciej Wozniak is an associated professor on Institute of Computer Science in AGH University of Science and Technology, Kraków, Poland. He received his master degree in Computer Science in 2013. He received his bachelor degree in Business Management in 2015. In 2017 he received his Ph.D. in computational science from AGH-UST. Since 2012 he is a member of Prof. Maciej Paszynski research group, working primarily on fast parallel direct solvers for isogeometric finite element methods targeting different parallel architectures.



Sergio Rojas is a professor of mathematics at Pontificia Universidad Católica de Valparaíso. He got his M.Sc. and Ph.D. in Engineering Sciences from Pontificia Universidad Católica de Chile in 2017, his M.Sc. in Mathematics from Pavia University in 2011, and his B.S. in Mathematics in 2008 from Pontificia Universidad Católica de Valparaíso. His research interests include numerical analysis, mathematical modeling, and scientific computing.